Chapter 3

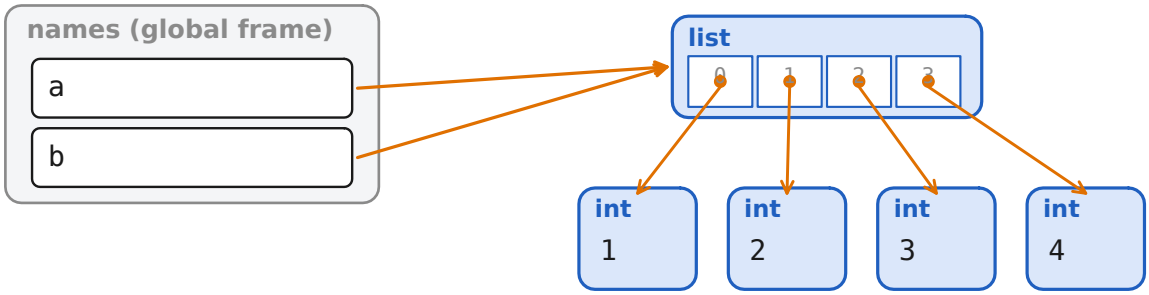
Names, objects and memory

names are labels on objects, not boxes holding values

Most "weird" Python behaviour comes from one wrong picture: thinking a variable is a box that **holds** a value. In Python a variable is a **name** that **refers to** an object living somewhere else in memory. Assignment never copies anything; it just points a name at an object.

```
1 a = [1, 2, 3]
2 b = a
3 b.append(4)
4 print(a, a is b, id(a) == id(b))
```

```
[1, 2, 3, 4] True True
```



There's **one** list and two names for it. `b.append(4)` changed the list, and `a` sees the change because `a` is the same list. Every picture in this chapter has this shape: names on the left, objects on the right, arrows for references.

> **KEY IDEA**
> `x = <expression>` means: evaluate the right side to an object, then make the name `x` refer to it. The name never contains the object.

Mutable vs immutable: the second half of the story

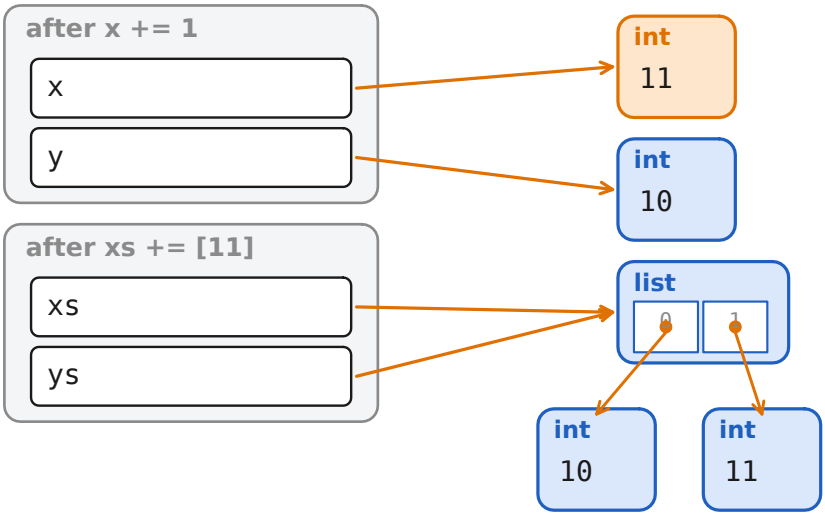
Some objects can change in place (**mutable**: list, dict, set, most class instances). Others can never change (**immutable**: int, float, str, tuple, frozenset, bytes). With an immutable object, "changing" a variable always means pointing the name at a **new** object:

```
1 x = 10
2 y = x
3 x += 1          # makes a NEW int 11 and rebinds x
4 print(x, y)
5
6 xs = [10]
7 ys = xs
8 xs += [11]      # list += mutates the SAME list in place
9 print(xs, ys)
```

← y still points at 10

← ys sees it

```
11 10
[10, 11] [10, 11]
```



immutable (safe to share)	mutable (sharing = aliasing)
`int`, `float`, `complex`, `bool`	`list`
`str`, `bytes`	`dict`, `set`, `bytearray`
`tuple` (but its items might be mutable)	instances of your classes (by default)
`frozenset`, `None`	`array.array`, NumPy arrays

TRAP

`x += 1` and `xs += [1]` look the same but do different things: for a list, `+=` is `xs.extend(...)` in place; for an int it builds a new object. `xs = xs + [1]` would build a new list and leave `ys` alone.

Passing arguments: the parameter is one more name

A function call doesn't copy arguments. The parameter becomes another name for the **same** object ("pass by assignment"). So a function can **mutate** what you pass in, but **rebinding** its parameter does nothing to the caller:

```python
def add_bonus(scores):
    scores.append(5)        # mutates the caller's list

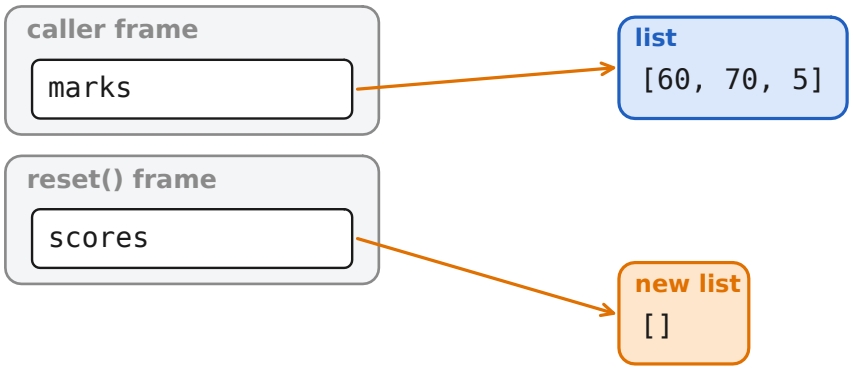
def reset(scores):
    scores = []             # rebinds the local name only

marks = [60, 70]
add_bonus(marks)
reset(marks)
print(marks)
```

← visible outside

← invisible outside

```
[60, 70, 5]
```



Inside `reset`, `scores = []` pointed the local name at a new empty list. The caller's list is untouched, and when `reset` returns, its frame and the empty list disappear.

> **RULE OF THUMB**
> A function should either **return a new value** or **mutate its argument and return None**, and its name should make clear which (`sorted(xs)` vs `xs.sort()`). Doing both confuses every caller.

Copies: shallow and deep

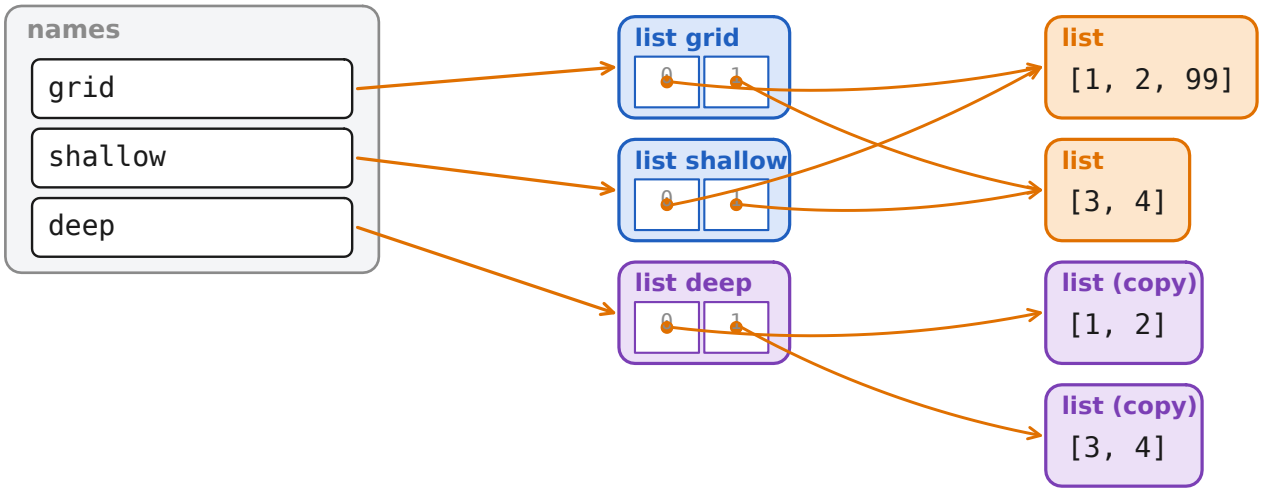
When you do want an independent object, copy it explicitly. A **shallow** copy makes a new outer container but reuses the items inside. For nested data that matters:

```python
import copy

grid = [[1, 2], [3, 4]]
shallow = grid.copy()           # same as list(grid) or grid[:]
deep = copy.deepcopy(grid)

grid[0].append(99)
print(shallow)                  # shares the inner lists
print(deep)                     # fully independent
```

```
[[1, 2, 99], [3, 4]]
[[1, 2], [3, 4]]
```



how	copies	inner objects
`b = a`	nothing (alias)	shared
`a.copy()`, `list(a)`, `a[:]`, `dict(d)`, `copy.copy(a)`	the outer container	shared
`copy.deepcopy(a)`	everything, recursively	new (sharing inside is preserved)

✎ YOUR GO (pencil, or the REPL, first)

Predict, then check: `m = [[0] * 3] * 3; m[1][1] = 5; print(m)`. Draw the boxes and arrows first.

Identity vs equality: is vs ==

`==` asks "same value?" by calling `__eq__`. `is` asks "same object?" by comparing identities (`id()`, in CPython the memory address). They differ more often than you'd think:

```
>>> a = [1, 2]; b = [1, 2]
>>> a == b, a is b
(True, False)
>>> x = 1000; y = int("1000")
>>> x == y, x is y
(True, False)
>>> s = None
>>> s is None
True
```

TRAP

Small ints (−5 to 256) and some strings are cached, so `is` sometimes "works" on them by accident. That's a CPython detail that changes between versions. Use `is` only for `None` (and other singletons like `True`/`False` or sentinel objects), and `==` for values.

Why lists can't be dictionary keys

A dict finds keys by their **hash**, computed once when the key is inserted. If a key could change after insertion, its hash would no longer match and the entry would be lost. So only immutable (hashable) objects can be keys or set members:

```
1 seen = {(1, 2): "tuple ok", "text": "str ok", frozenset({3}): "frozenset ok"}
2 print(len(seen))
3 seen[[1, 2]] = "list?"
```

```
3
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

```
seen[(1, [2])] = "tuple holding a list?"
```

```
TypeError: cannot use 'tuple' as a dict key (unhashable type: 'list')
```

How long objects live

CPython keeps a **reference count** on every object: how many names, list slots, attributes etc. point at it. When the count drops to zero the object is freed immediately. `del x` removes a **name**; the object only dies if that was the last reference:

```
import sys

data = [0] * 1000
alias = data
print(sys.getrefcount(data) - 1)   # minus the temporary ref from the call itself
del data
print(len(alias))                  # still alive: alias refers to it
```

```
2
1000
```

Reference counting can't free **cycles** (two objects pointing at each other), so a separate garbage collector (`gc`) finds and frees unreachable cycles from time to time:

```
import gc

class Node:
    def __init__(self):
        self.other = None

a, b = Node(), Node()
a.other, b.other = b, a       # a cycle
del a, b                      # unreachable, but refcounts are still 1
print(gc.collect() >= 2)      # the cycle collector frees them
```

```
True
```

size of...	on 64-bit CPython	why
`sys.getsizeof(0)`	28 bytes	every int is a full object (type pointer, refcount, digits)
`sys.getsizeof([])`	56 bytes	header plus room to grow
a list of 1,000 ints	~8 KB of pointers + 28 KB of int objects	the list holds references, not values
an `array.array` of 1,000 ints	~4 KB	raw 4-byte values packed together; NumPy does the same (Ch 18)

IN OTHER LANGUAGES

C has no references by default: `int x = y;` really copies, and you manage memory yourself. Java works like Python for objects (references) but copies primitives (`int`). Rust makes ownership explicit: assignment **moves** a value unless you borrow it. The passport question "what does assignment do?" has a different answer in each.

☐ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 3.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 3.1` shows other valid solutions and their trade-offs.

`3.1` fix the bug The function that changes your data
`3.2` build from a spec Write your own deep copy
`3.3` learn an unseen module Spot the aliasing (id and is)
`3.4` build from a spec Make it hashable
`3.5` mini-project Mini-project: undo for a document

PASSPORT STAMP ask the same question of every new language (Appendix B)

2. Memory model

Every value is an object on the heap; variables, list slots and attributes are references. Assignment and argument passing copy references, never objects. Freed by reference counting, plus a cycle collector.

3. Typing

Values have types; names don't. Mutable (list, dict, set, instances) vs immutable (int, str, tuple, frozenset); only hashable objects can be dict keys.

DEEP DIVE (your vault)

Vault notes: 5.3 Objects: identity, type, and value · 5.4 Names and bindings, not variable boxes · 5.6 References, reachability, and lifetime · 5.7 Mutability and shallow structure · 5.8 Equality, ordering, and hashing · 8.3 Python passes object references by assignment · 15.12 Reference counting · 15.13 Cyclic garbage collection.